Cursor Co-pilot

Full Feature Guide and Operating Instructions

Generated 2026-06-15 from the local project workspace.

Cursor Co-pilot is a context-aware assistant for Cursor that combines an MCP server, local project scanner, official Cursor documentation retrieval, MCP recommendations, rule/config scaffolding, a web dashboard, and a native floating desktop companion.

Executive Summary

- Primary mode: a Cursor MCP server named cursor-copilot with 11 callable tools.
- Desktop mode: a transparent always-on-top mascot launched by run_desktop_companion.bat.
- Project intelligence: scans manifests, file tree, README, Cursor rules, and Jupyter notebooks.
- Knowledge layer: BM25 retrieval over Cursor docs plus MCP catalog data.
- LLM layer: Anthropic, Gemini, and Ollama fallback chain with full backend error reporting.
- QA surface: health, project status, chat SSE, and MCP stdio smoke tests.

System Architecture

Component	Files	Role
MCP server	mcp_server.py	Exposes tools over stdio so Cursor and other MCP clients can call project-aware capabilities.
LLM backend	llm.py	Selects Anthropic, Gemini, or Ollama; falls through usable backends and reports the full failure chain.
Project scanner	scanner.py	Infers language, framework, package manager, SDKs, file summaries, and notebook health.
Retrieval and ingestion	search.py, ingest.py	Builds and queries a BM25 index over Cursor docs and the MCP catalog.
Artifact generation	cursor_artifacts.py	Renders Cursor rules and MCP config snippets.
Web dashboard	app.py, ui.py	Serves chat, project status, install/rule APIs, and health checks.

Component	Files	Role
Desktop companion	companion.py	Native Tkinter overlay with drag, chat, dashboard open, and notebook warnings.
Desktop launcher	desktop_launcher.py, run_desktop_companion.bat	Starts or reuses the backend, waits for health, and launches the companion.

Launch and Use Modes

Recommended desktop launch

```
run_desktop_companion.bat
```

- Reuses a healthy backend on http://127.0.0.1:59712 when one is already running.
- Starts app.py on port 59712 when no backend is available.
- Sets OPEN_BROWSER=0 so the web dashboard does not steal focus.
- Launches companion.py after the health endpoint responds.
- Legacy run_companion.bat forwards to the same orchestrated launcher.

Floating companion interactions

- Left-click and drag to reposition the mascot.
- Right-click to open a small chat box and ask a question.
- Double-click to open the full dashboard.
- On launch, the companion calls /api/project/status and warns about Jupyter notebooks with out-of-order execution.
- A socket lock prevents duplicate companion widgets.

Web dashboard

```
python app.py
```

The dashboard starts on a random free port by default and opens a browser. Set PORT to pin the port and OPEN_BROWSER=0 to suppress browser auto-open.

Local API usage

```
Invoke-RestMethod -Uri "http://127.0.0.1:59712/api/health"

Invoke-RestMethod -Uri "http://127.0.0.1:59712/api/project/status"
```

The chat endpoint streams Server-Sent Events. Project-aware questions such as stack scans and notebook health include local scanner context before the LLM answers.

MCP Server Features

Tool	Purpose	LLM
ask_cursor_docs	Answer Cursor feature questions grounded in official Cursor docs.	Yes
search_cursor_docs	Return ranked Cursor documentation chunks without synthesis.	No
scan_project	Scan a project for language, framework, package manager, SDKs, files, README, and notebook data.	No

Tool	Purpose	LLM
suggest_mcp_servers	Recommend MCP servers or skills from the indexed catalog for a goal and optional project scan.	Optional
generate_cursor_rule	Generate .cursor/rules/*.mdc content for a behavior or convention.	Yes
scaffold_mcp_config	Create merge-ready .cursor/mcp.json snippets from catalog server names.	No
recommend_setup	One-call setup advisor: scan project, recommend servers, produce snippets and tips.	Yes
create_cursor_rule	Generate and write a Cursor project rule file directly to disk.	Yes
install_mcp_server	Install or merge a catalog MCP server into project/global MCP config.	No
scan_notebooks	Report Jupyter notebook imports, variables, headers, execution counts, and order health.	No
search_knowledge_vault	Search personal Markdown or Obsidian notes with BM25 ranking.	No

Project Intelligence

- Detects Python, JavaScript/TypeScript, Go, Rust, Java, and other languages from file extensions.
- Reads package manifests such as requirements.txt and package.json.
- Infers frameworks such as Flask, FastAPI, Django, React, Next.js, Express, and others from dependencies.
- Detects notable SDKs and services such as Anthropic, Gemini/OpenAI-related libraries, Stripe, Supabase, AWS, Redis, Postgres, and ML/data libraries.
- Scans Jupyter notebooks for code/markdown cell counts, imports, variables, headers, execution counts, and out-of-order execution.
- Injects local project context into chat prompts for stack, project scan, Jupyter, and notebook questions.

Example project-aware prompts

```
Co-pilot, scan my project and tell me what stack we are using.

Does my Jupyter Notebook have any out-of-order execution issues?

Recommend MCP servers for this Python Flask project.

Create a Cursor rule for this repository's Python conventions.
```

LLM and Retrieval Behavior

- Cursor docs answers are grounded in retrieved documentation chunks.
- Retrieval uses a local BM25 index built from Cursor docs and MCP catalog sources.
- LLM preference is configured with LLM_BACKEND; supported values are anthropic, gemini, and ollama.
- Backend resolution moves the preferred backend to the front, then falls through Anthropic, Gemini, and Ollama order.
- When all backends fail, the user-facing error reports every backend failure in order.
- Project/status API redacts secret-like MCP environment values before returning config data.

Configuration

Variable	Default	Meaning
ANTHROPIC_API_KEY	unset	Enables Claude backend.
ANTHROPIC_MODEL	claude-opus-4-8	Claude model selection.
GEMINI_API_KEY	unset	Enables Gemini backend.
GEMINI_MODEL	gemini-2.5-flash	Gemini model selection.
LLM_BACKEND	anthropic	Preferred backend; moved to the front of the fallback chain.
OLLAMA_MODEL	gemma3:4b	Local model name for Ollama.
OLLAMA_URL	http://localhost:11434	Ollama server URL.

Variable	Default	Meaning
TOP_K	5	Number of doc chunks retrieved for web chat.
PORT	random	Forces the web UI/API port.
OPEN_BROWSER	1	Set to 0 to suppress browser auto-open.

Cursor and External Editor Integration

Cursor project MCP config

```
{
"mcpServers": {
"cursor-copilot": {
"command": "C:/path/to/.venv/Scripts/python.exe",
"args": ["C:/path/to/cursor-co-pilot/mcp_server.py"],
"env": {
"GEMINI_API_KEY": "${env:GEMINI_API_KEY}",
"LLM_BACKEND": "gemini",
"GEMINI_MODEL": "gemini-2.5-flash"
}
}
}
}
```

- In Cursor, enable the server from Cursor Settings > Features > MCP.
- Use an absolute venv python path on Windows so Cursor runs the correct interpreter.
- The same stdio MCP server can be registered in other MCP-capable editors such as Cline, Roo-Code, Windsurf, or Claude Desktop.
- Do not place literal API keys in docs or shared config examples; use environment placeholders.

Superpowers Workflow

```
/using-superpowers
/using-superpowers QA this companion integration end to end.
/using-superpowers Write an implementation plan before changing behavior.
/using-superpowers Debug why chat fallback is not using Gemini.
```

- Use brainstorming before creative feature work or behavior changes.
- Use writing-plans before multi-step implementation.
- Use test-driven-development for bug fixes and behavior changes.
- Use systematic-debugging before fixes when behavior is unexpected.
- Use verification-before-completion before claiming anything is fixed or complete.

QA Checklist

```
.\.venv\Scripts\python.exe -m unittest discover -v

.\.venv\Scripts\python.exe -m py_compile app.py llm.py desktop_launcher.py companion.py
mcp_server.py

Invoke-RestMethod -Uri "http://127.0.0.1:59712/api/health"

.\run_desktop_companion.bat
```

- Health should report chunks > 0 and an active LLM backend.
- Launcher should say Backend already healthy when the backend is already running.
- Stack chat should mention Python, Flask, pip, SDKs, and notebook health for this project.
- MCP smoke test should list 11 tools and confirm scan_project plus scan_notebooks.
- Project status should redact GEMINI_API_KEY and other secret-like env values.

Troubleshooting

Symptom	Likely Cause	Action
MCP server will not connect	Wrong Python path, missing deps, stdout pollution.	Use absolute venv python path, install requirements, keep MCP logs on stderr.
Gemini 429 RESOURCE_EXHAUSTED	Key is valid but credits or billing are depleted.	Add credits, use another key, configure Anthropic, or start Ollama.
Chat says local model unreachable	Ollama fallback is selected or reached but not running.	Run ollama serve and pull the model, or configure a cloud backend.
Companion launches but no answer	Backend unavailable or every LLM backend failed.	Check /api/health and the SSE chat error chain.
Dashboard steals focus	app.py launched directly with browser auto-open.	Use run_desktop_companion.bat or set OPEN_BROWSER=0.
Notebook warning appears	Scanner found out-of-order execution counts.	Re-run notebook cells in order or clear outputs/execution counts.

Verified Current State

- Desktop launcher reboot/reuse path verified.
- Backend health verified on port 59712 with Gemini active and 191 indexed chunks.
- Live project-aware chat verified for stack and notebook health questions.
- Unit suite verified with 13 passing tests.
- Python compile check verified for app.py, llm.py, desktop_launcher.py, companion.py, and mcp_server.py.
- Linter diagnostics reported no errors for touched files.